AI Engineering and Claude Architect · Part IV, Java

AI in Java, Lesson 4

Building an MCP Server in Java

AI06 L4 taught that the current MCP protocol is stateless, with no handshake. Then this module's Java client connected, and negotiated version 2025-11-25, with an `initialize` call. Neither source was wrong. The Java SDK just hadn't caught up.

Module	AI08 of 13
Lesson	4 of 8
Prerequisites	AI06 L4–L5, AI08 L1
Exam relevance	Java is out of scope. Implements TS 2.2 and TS 2.4 ideas
Reading time	~30 minutes

Contents

1. Building an MCP Server in Java

Lesson 4 of 8 · AI in Java

Building an MCP Server in Java

Key idea

The official Java SDK gives you a working MCP server in a page of code, served from your existing Spring Boot app. The server is the easy part. **The work is deciding what should be a server at all, securing it, and checking which protocol version your SDK really speaks, because SDKs adopt new versions on their own schedules.**

> *Before: MCP's concepts and errors (AI06 L4–L5). Teaches: a real server on the official SDK over Streamable HTTP, a real client round trip, and the security you must add. Used by: AI08 L8, AI09 L3.*

The hook: a real round trip

The official SDK's client calls our server over real Streamable HTTP on localhost. Real output from `McpServerTest`:

```
protocol version: 2025-11-25
server: orders
tools: [lookup_order]
resource: [TextResourceContents[uri=orders://schema, mimeType=text/plain,
           text=order_id, status, total, delivered_on, meta=null]]
call 7781: isError=false Order 7781: delivered, total $40
call 9999: isError=false No order 9999 exists
call abc: isError=true {"errorCategory":"validation","isRetryable":false,
                        "message":"order_id must be digits only"}
unknown tool: McpError: Unknown tool: invalid_tool_name
```

> *Key idea*
>
> **SDK 2.1.0 negotiated protocol version 2025-11-25, not 2026-07-28, which AI06 L4 described.** So it opens with `initialize`, and it isn't the stateless model. The MCP docs say that SDKs "adopt new protocol revisions on their own schedules". The Java SDK is Tier 2. Check this before assuming a feature exists. I tested only this SDK talking to itself.

One oddity: the unknown-tool error text said `invalid_tool_name`, rather than the name I called (`delete_order`). The error kind is right, and the message isn't helpful, so log the requested name yourself.

Intuition: opening a service counter

Your business already works. Opening a counter that outsiders can use doesn't change what you do, but now you need a clear menu (tools), a brochure rack (resources), an ID check, and limits on who can ask for what. Those last two are what teams forget.

Mechanism: the server

From `OrdersMcpServer.java`. It's plain SDK code, in a Spring configuration class.

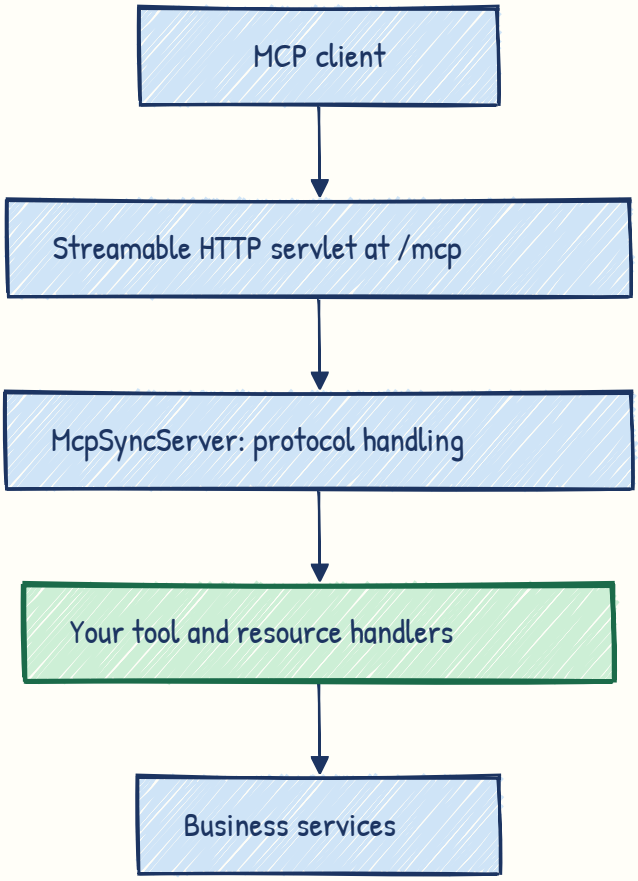


Figure 1. How the pieces connect. The transport is a servlet in the existing app, the SDK handles the protocol, and your handlers hold the logic.

The transport, as a normal servlet bean, per the SDK docs:

```
return HttpServletStreamableServerTransportProvider.builder()
        .jsonMapper(McpJsonDefaults.getMapper())
        .mcpEndpoint("/mcp")
        .build();
```

The server and its capabilities:

```
McpSyncServer server = McpServer.sync(transport)
        .serverInfo("orders", "1.0.0")
```

A tool handler. Bad input returns `isError`, and no match is a normal result:

```
if (!id.matches("\\d+")) {
    return result("{\"errorCategory\":\"validation\","
```

```
        return result(found == null ? "No order " + id + " exists"
                : "Order " + id + ": " + found, false);
```

Outcome	How to report it	Who can act
Success, or nothing found	Normal result	The model
Bad input, business rule	`isError(true)`, with a useful message	The model can correct itself
Infrastructure failure	Throw `McpError`	The client

That follows the SDK's docs: "Use `isError(true)` for validation and domain failures. Reserve thrown exceptions for unexpected infrastructure failures." The docs also say input validation against `inputSchema` is on by default.

A resource, for a catalog (TS 2.4):

```
        McpSchema.Resource.builder()
                .uri("orders://schema")
```

Spring integration, and a version catch

Route	What the docs say	Fit for Boot 3
Servlet transport in core	Included in the core SDK, with no web framework	**Used here**
`mcp-spring-webmvc`, `mcp-spring-webflux`	Now ship with Spring AI 2.0+	Spring AI 2.0 targets Boot 4
Spring AI MCP Boot starter	Higher level, with annotations	Check for your Spring AI line

Jackson note: the SDK pulled in `mcp-json-jackson3`, so this project has Jackson 2 from Boot, and Jackson 3 from MCP, side by side. They use different packages, and the tests pass. Know that it's there, since it adds weight.

Connecting a client

In `.mcp.json` (AI06 L5). **I haven't tested this from Claude Code**, so it's the expected shape only:

```
{
  "mcpServers": {
    "orders": {
      "type": "http",
      "url": "${ORDERS_MCP_URL:-http://localhost:8080}/mcp",
      "headers": { "Authorization": "Bearer ${ORDERS_MCP_TOKEN}" }
    }
  }
}
```

Security you must add

This server has no authentication. That's fine for a test on localhost, and for nothing else.

Control	Why	Note
Origin and Host validation	Blocks browser-based attacks on local servers	**Off by default**, per the SDK docs. Enable with `httpHeaderValidator`
Authentication	Who's calling?	OAuth is recommended for remote servers
Per-user authorization	The tool must check the caller's rights	A valid token doesn't mean access to every order
Input validation	Spec says servers must validate	On by default in the SDK. Add business checks
Rate limits	Agents can loop	Spec says servers must rate limit
Output sanitizing	Results go into a model's prompt	Treat them as a possible injection path (AI09 L2)
Annotations	Mark destructive tools	Clients must treat them as untrusted

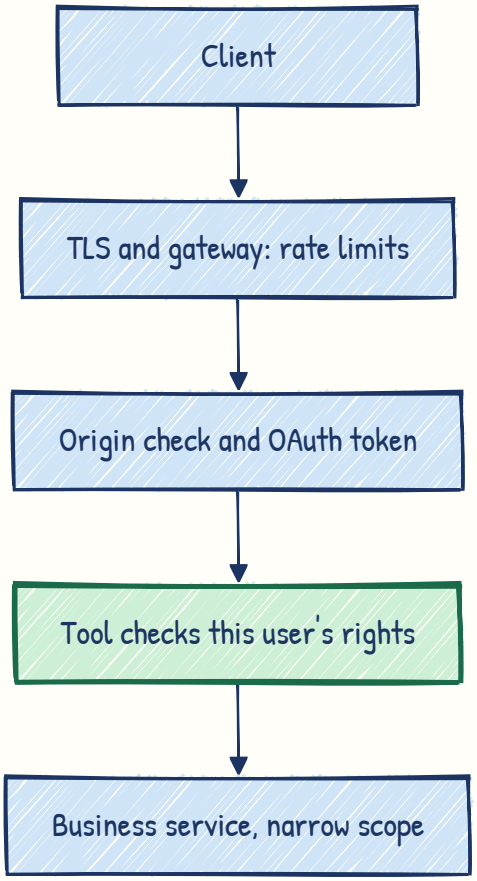


Figure 2. Security layers for a remote server. Most of this sits in front of or around the SDK, rather than in it.

Why the obvious fixes fall short

Tempting move	Why it fails	Better
Wrap each REST endpoint as a tool	Not shaped for agents (AI06 L3)	Task-shaped tools
Use one shared service token	Every user gets the server's rights	Pass user identity through
Throw for validation failures	The model can't self-correct	`isError`
Build a server when one agent needs one function	Extra moving part	A plain tool
Assume the latest protocol	SDKs lag	Check what's negotiated
Custom server for Jira	You maintain it	Use an existing one

How Anthropic and the field approach it

Point	Source
Official SDK list, with Java at Tier 2	MCP SDK page
Server, transports, tools, errors, resources	MCP Java SDK server docs
Servers must validate inputs, control access, rate limit, and sanitize outputs	MCP tools spec
A deterministic tool order helps prompt caching	Same

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: a server on localhost, with no auth.

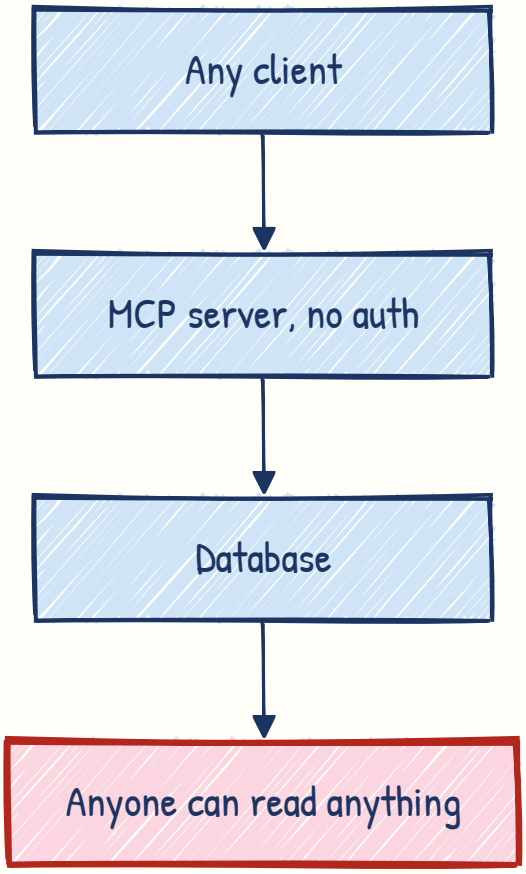


Figure 3. Baseline. A working demo with no identity and no limits, reachable by anything that finds it.

Improved: a shared token. Only approved clients connect. New problem: **the server acts with one identity, so user A's agent can read user B's orders.**

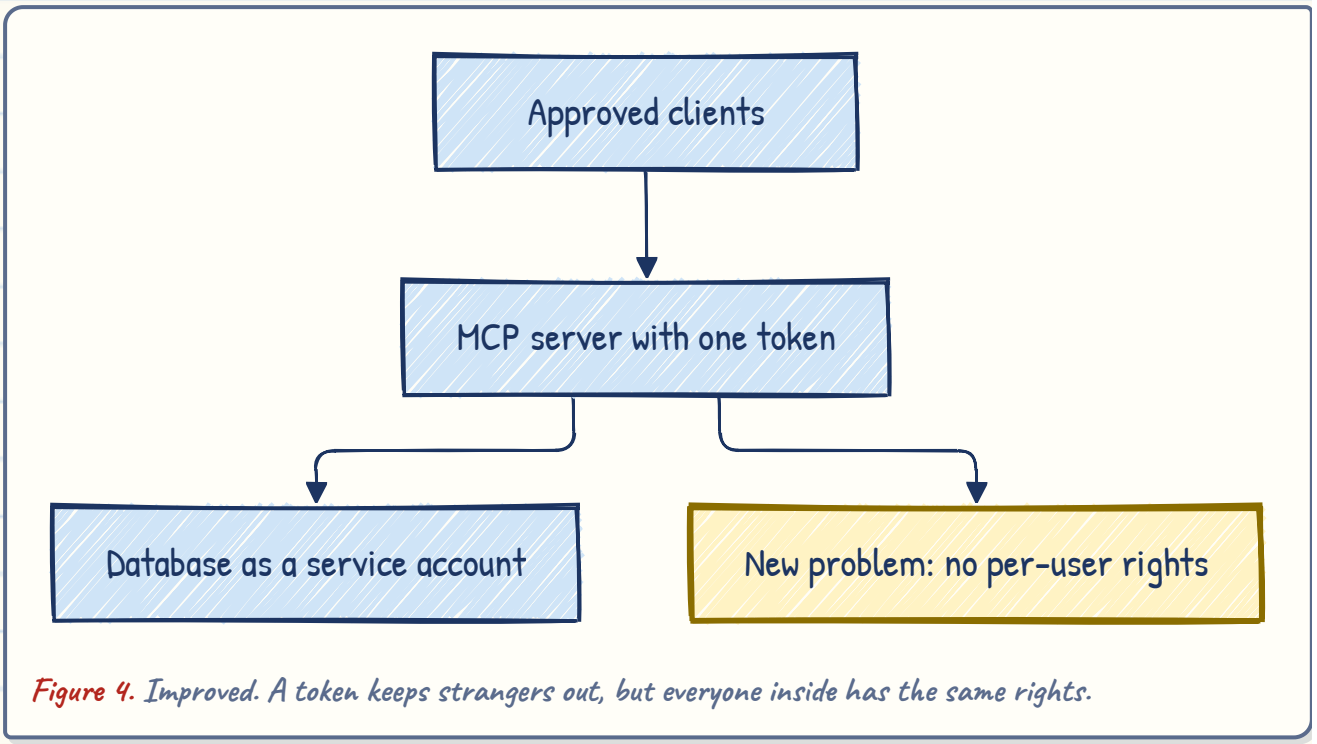


Figure 4. Improved. A token keeps strangers out, but everyone inside has the same rights.

Production: user identity through to the data.

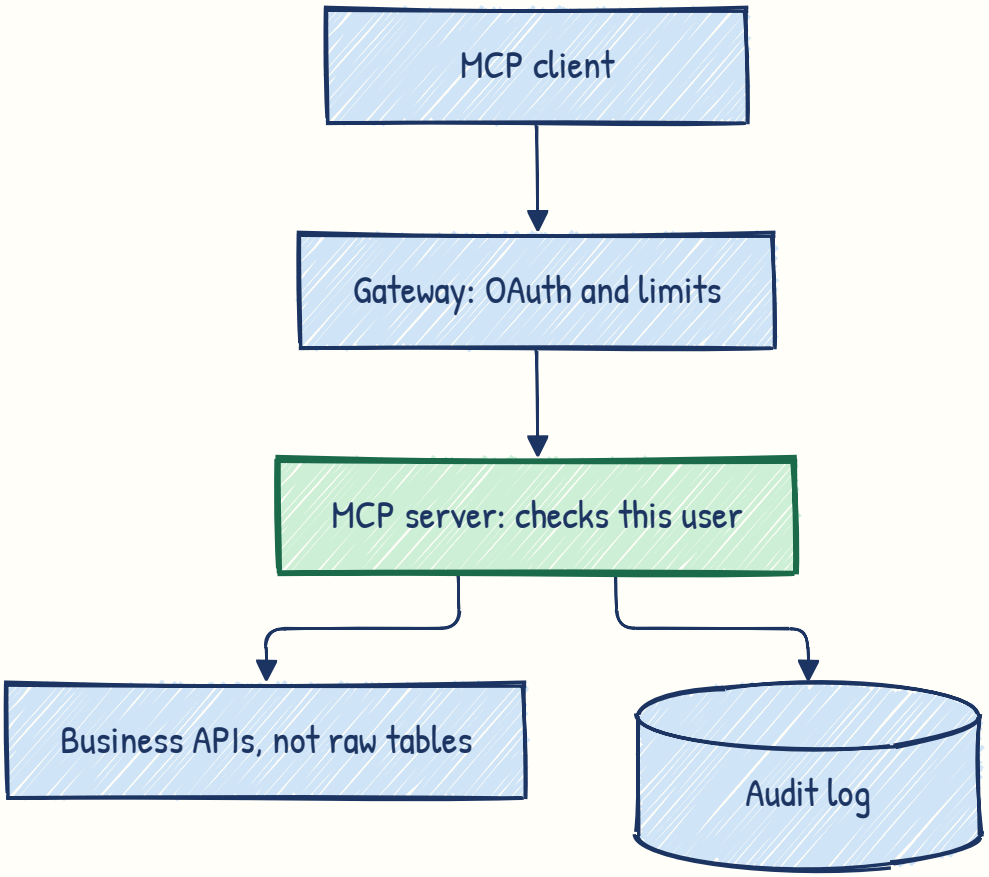


Figure 5. Production. A gateway authenticates and limits, the server authorizes per user, and the backend enforces its own rules. All calls are audited.

Failures

Failure	Containment
Backend down	`isError`, transient, or `McpError` for infrastructure failure
Client and server versions differ	Negotiation, and test the pair you ship
Tool list changes	Deterministic order, and evals on change
Loops hammer a backend	Rate limit and circuit breaker
Injection through tool output	Treat as data (AI09 L2)
Session lost, on this stateful protocol	Sticky routing or a shared session store. Check what your version needs

Trade-offs

Decision	Gain	Cost
Inside the existing app	Reuses services and deployment	Shared failure and scale
Separate service	Independent scale and security	Another deployment
Sync server	Simple code	One thread per call, which virtual threads help with
Official SDK	Follows the spec	Lags the newest protocol

When not to build one: one agent, one function, or a standard system that has a server already.

Common mistakes

1. **Shipping with no auth.** Origin checks are off by default too.
2. **One service identity for all users.** Authorize per user.
3. **Throwing for validation errors.** Use `isError`.
4. **Assuming the newest protocol.** Check what's negotiated.
5. **Exposing raw tables.** Expose task-shaped tools over business APIs.
6. **Trusting tool annotations.** Clients mustn't, and you shouldn't assume they don't.
7. **Testing only with your own client.** Try the client you'll actually use.

What to say in an interview

"With the official Java SDK, the server itself is about a page of code, as a servlet in Spring Boot. The real work is security: origin checks are off by default, auth is on me, and a valid token doesn't mean rights to every record, so tools authorize per user. I'd also check the negotiated protocol, since my test with SDK 2.1.0 used 2025-11-25, which isn't the latest spec."

Certification lens

> *How this lesson maps to CCAR-F*
>
> **Java isn't tested.** `isError` and catalogs as resources are (TS 2.2, TS 2.4).

These are original practice questions for understanding, not real or leaked exam questions.

Q Q1 · Code analysis · Medium. A handler does `throw new RuntimeException("date must be YYYY-MM-DD")`. What's wrong, and how would you rewrite it?

A It reports a fixable input problem as an infrastructure failure, so the model may not get a chance to correct itself. Return a `CallToolResult` with `isError(true)`, and a message showing the right format. Throw only for unexpected failures.

Q Q2 · Security · Hard. Your internal MCP server has a valid OAuth flow, and any employee can use its `get_payslip(employee_id)` tool. What's wrong, and what's the fix?

A Authentication isn't authorization, and the ID is model-supplied. Anyone, or an injected prompt, could read anyone's payslip. Take identity from the token, drop the parameter, and enforce access in the backend. Audit each call, and test the no-access case.

Q Q3 · Troubleshooting · Medium. Your team reads that the latest MCP spec is stateless, and designs load balancing on that basis. Traffic through the Java server shows per-client sessions. What's going on?

A SDKs adopt new revisions on their own schedules. Check the negotiated version: in this module's test, SDK 2.1.0 used 2025-11-25 with `initialize`. So design for what you actually run, which may need sticky routing or a shared session store, and retest on upgrade.

Revision - Lesson 4

→ The official Java SDK is Tier 2, and version 2.1.0 is its latest stable release.

→ Streamable HTTP runs as a servlet in your existing app.

→ The test negotiated protocol 2025-11-25, with initialize, not the 2026-07-28 model.

→ SDKs adopt new revisions on their own schedules, so check.

→ isError for validation and domain failures. McpError for infrastructure.

→ No match is a normal result.

→ Spring-specific transports now ship with Spring AI 2.0+.

→ Origin and Host validation is off by default.

→ Authentication isn't authorization: check per user, in the tool and backend.

→ Expose task-shaped tools over business APIs, not tables.

→ Rate limit, and treat tool output as untrusted.

Your notes